

Yitae Jeong

AI Agent Platform / LLM Infrastructure Engineer

jeongiitae6@gmail.com

TECHNICAL PROFILE

Builder-oriented LLM engineer focused on agent platform infrastructure, Graph-RAG runtimes, evaluation loops, and on-prem inference systems. Designed and benchmarked SEOCHO, an open source agentic Graph-RAG runtime that turns documents into graph-backed evidence, routes queries through specialist agents, and exposes traceable answer contracts. Comfortable working from frontier idea to production-shaped implementation: define the failure mode, instrument it, benchmark alternatives, then turn the result into a reusable runtime contract, provider gateway configuration, CLI/workflow, guardrail, or platform capability. Heavy user of Codex and Claude Code, with AI-assisted development kept disciplined through ``.beads``, GitHub Actions, CI gates, and reviewable automation.

WHY THIS FITS AGENT FLEET INFRASTRUCTURE

- **Skill-style platform thinking:** built modular agent/runtime contracts around ontology profiles, evidence bundles, route profiles, answer-shape metadata, and graph-specialist paths; directly transferable to OpenClaw-style reusable skill units and registries.
- **Harness and lifecycle engineering:** worked on semantic routing, agent readiness, tool-budgeted execution, graph target selection, evidence assembly, conservative abstention, and traceable answer synthesis.
- **Telemetry-driven optimization:** used JSONL and Opik-style traces, fixed-answerer comparisons, paired tests, latency/cost metadata, and failure-mode inspection to convert logs into platform decisions.
- **AI-native developer workflow:** ran SEOCHO development with Codex, Claude Code, ``.beads`` task tracking, repo-local scripts, GitHub Actions, scheduled automation prompts, and reviewable PR gates.
- **LLM infrastructure depth:** recent work includes vLLM, LMCache, Blackwell Pro-class setup, KV-cache offloading, RDMA/NIXL/UCX/RoCEv2, NIC tuning, and TTFT-focused workload analysis.
- **Multi-provider orchestration:** built OpenAI-compatible provider abstractions across OpenAI, DeepSeek, Kimi/Moonshot, Grok, Qwen, and MARA Cloud, including provider capability specs, base URLs, env-key resolution, timeout/retry policy, and usage/latency metadata.
- **Security and reliability instinct:** used ontology guardrails, context graphs, provenance, missing-slot visibility, policy-aware runtime contracts, and no-silent-fallback rules to reduce unsupported answers and API-model risk.

SELECTED ENGINEERING RESULTS

Agent Operations / Developer Workflow

- Operated ``.beads`` as the execution source of truth for tasks, dependencies, status, and landing discipline, similar to an internal agent-ops control plane.
- Built GitHub/Codex automation surfaces for scheduled maintenance, periodic review, CI validation, docs checks, and draft PR workflows.
- Maintained ``.seocho.blog`` as an in-repo Astro/Starlight docs and blog site, generated from canonical repo docs and deployed through GitHub Pages.
- Added developer-facing CLI/workflow surfaces for provider selection, graph routing hints, local query/index paths, tracing, and runtime inspection.

Agent Platform / Graph-RAG Runtime

- Built SEOCHO as an API-based agentic Graph-RAG runtime: ingestion, chunking, entity extraction, graph writes, semantic routing, evidence assembly, answer synthesis, and tracing.
- Ran a FinDER-style retrieval study on **910 financial QA rows** across six slices, comparing vector, graph, and vector+graph retrieval with ontology size as a controlled variable.
- Ran BC3 decision-QA experiments across **135 cases x 5 lanes = 675 comparison runs**; results changed the product direction from "graph always wins" to route-by-query and evidence-first answering.
- Added route/evidence contracts so required slots, missing slots, provenance, route profile, and answer shape remain visible before answer synthesis.
- Implemented vendor-neutral observability with JSONL as the portable trace artifact and Opik as the optional team dashboard/exporter.
- Prepared KGC2026 material using SEOCHO to explain graph-agentic RAG across RDF/LPG specialist agents, ontology guardrails, and evidence-grounded answers.

Inference / Cost / Throughput

- Orchestrated MARA Cloud as an OpenAI-compatible multi-model gateway for DeepSeek-V3.1, MiniMax-M2.5, and gpt-oss-120b, using it for generator and LLM-as-judge workloads in benchmark pipelines.
- Measured KV-cache cost behavior on BC3: graph context reached a **59% cached-token hit ratio** vs vector **0%**, with lower amortized billable prompt tokens despite **2.2x** larger raw context.
- Worked on vLLM + LMCache KV offloading and reuse-vs-recompute decisions to reduce recomputation pressure and improve first-token latency.
- Worked on multi-node serving prerequisites: RDMA setup, NIXL, UCX, RoCEv2, and NIC-engine tuning for high-throughput KV movement and cache offload paths.
- Profiled DozerDB/Neo4j graph hydration and adopted a Rust PackStream codec: **1.62x** faster real 3-shard federation read and **3.57x** faster 130k-row bulk hydration with hash-parity checks.

- Prepared KCC2026 tutorial material on inference workloads, KV-cache footprint, vLLM internals, LMCache data paths, scheduler-side decisions, and worker-side KV movement.

ENGINEERING PROJECTS

SEOCHO: Agentic Graph-RAG Runtime and Evaluation Platform

Open source, 2026-present | GitHub: github.com/tteon/seoch0 | DeepWiki: deepwiki.com/tteon/seoch0

- Implemented API-side ingestion-to-answer flows with graph-backed memory, ontology context, evidence bundles, and answer guardrails.
- Built semantic routing across LPG/RDF graph-specialist paths, answer generation, graph reasoning guardrails, and inspectable evidence contracts.
- Converted experiment findings into product contracts: ontology profile selection, route profiles, answer-shape metadata, missing-slot guardrails, and stable trace payloads.
- Built and maintained developer workflows around `.beads``, repo-local scripts, GitHub Actions, Codex automation prompts, and CI quality gates.
- Used external foundation-model APIs with context-graph and ontology guardrails, reflecting how to build reliable agent systems without owning the base model.
- Added spec-driven LLM backend configuration for OpenAI-compatible providers, including MARA Cloud gateway support, model capability resolution, provider request quirks, and explicit retry/timeout control.
- Benchmarked vector, graph, and hybrid retrieval lanes with fixed answerer controls, paired comparisons, persisted traces, and failure-state inspection.
- Owned the public documentation surface through ``seoch0.blog``, an Astro/Starlight site generated from repository docs and deployed via GitHub Pages.

On-Prem LLM Inference Infrastructure

Current work, 2026-present | Multi-node serving infrastructure and workload analysis

- Set up server infrastructure for LLM inference workloads, including Blackwell Pro-class hardware preparation and software stack bring-up.
- Built a vLLM + LMCache serving stack to study KV-cache reuse, prefix caching, chunked prefill, scheduler behavior, and tiered cache design.
- Analyzed ISL, OSL, prefill/decode ratio, TTFT, ITL/TPOT, E2E latency, throughput, cache hit ratio, GPU memory pressure, and network transfer costs.
- Evaluated KV offloading and reuse-vs-recompute tradeoffs using KV size, transfer bandwidth, cache-tier latency, prompt length, and workload locality.

TECHNICAL SKILLS

Agent Platform: skill-style contracts, tool execution, semantic routing, evidence bundles, guardrails

Provider Routing: MARA Cloud, OpenAI-compatible gateways, multi-provider configs, retry/timeout policy

AI-Native Workflow: Codex, Claude Code, `.beads``, GitHub Actions, CI gates, PR automation

RAG/Agents: Graph-RAG, RDF/LPG agents, vector/hybrid retrieval, grounded answers, tool calling

LLM Infra: vLLM, LMCache, KV offload, token budgeting, context management, TTFT/ITL analysis

Observability: JSONL traces, Opik, OTel-style trace contracts, paired tests, latency/cost metadata

Serving Infra: RDMA, NIXL, UCX, RoCEv2, NIC tuning, GPU server setup, cache tiering

Stack: Python, FastAPI, OpenAI Agents SDK, DozerDB/Neo4j, Astro/Starlight, CLI/dev workflows

SELECTED LINKS

SEOCHO GitHub: <https://github.com/tteon/seoch0>

SEOCHO DeepWiki: <https://deepwiki.com/tteon/seoch0>